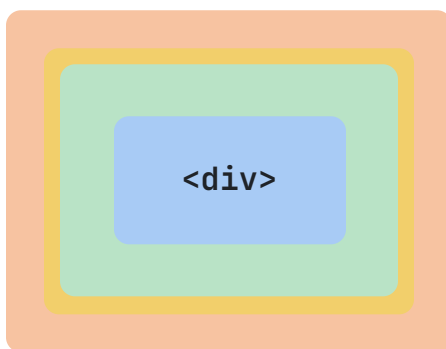


名前がわかれば、AIに頼める。

AIへの指示が 一発で通る Webサイト用語 図解帳

「ボタンの **padding** を
上下16px・左右24pxにして」



いつものアプリも、中身はWebページ

チャットやメモのアプリも、Webページと同じ部品でできている。



画面の部品には名前がある

- 14 押すと開いたり閉じたりする
アコーディオン
- 15 押すと中身が切り替わる
タブ
- 17 少しだけ出て自動で消える
トースト

番号は、この本で図解している用語の番号。

Slack や Notion などのパソコン用アプリは、じつはWebページをアプリの形に包んだもの。アプリにするときは **Electron (エレクトロン)** という道具を使う。だから部品も、Webサイトと同じ。

こんな場面で、名前が効く

自分用アプリを作る 「日報を書く小さなアプリが欲しい」とAIに頼めば、Electron を使って**パソコン用アプリ**にしてくれる。画面の注文は、この本の言葉でできる。

不具合を伝える ~~「左の、開くやつが動かない」~~
→ 「左の**アコーディオン**が開かない」

社内ツールを直す ~~「保存できたかわからない」~~
→ 「保存したら**トースト**で知らせて」

試してみよう Chrome でいつものサイトを開き、右クリック → [検証]。画面の横に、そのページの部品の一覧が出てくる。部品にマウスを乗せると、画面のどこかが光る。



01	要素 (element)	4	11	ドロワー	14
02	タグとclass	5	12	パンくずリスト	15
03	ボックスモデル	6	13	モーダル	16
04	padding	7	14	アコーディオン	17
05	margin	8	15	タブ	18
06	border	9	16	カルーセル	19
07	ブロックとインライン	10	17	トースト	20
08	ヘッダー・フッター	11	18	ファーストビュー	21
09	グローバルナビ	12	19	レスポンスとブレイクポイント	22
10	ハンバーガーメニュー	13	20	CTA	23

付録	AIを使いこなす知恵 (名前の聞き方／指示の組み立て方／モックから本番へ)	24
----	---------------------------------------	----

01 要素

ページを組み立てる部品1個分。タグで囲んだひとかたまり。

① 季節のタルト

② 旬の果物をたっぷりのせました。

③

④ 予約する

① <h1>季節のタルト</h1>

② <p>旬の果物を...</p>

③

④ <button>予約する</button>

画面の部品1つが、HTMLの要素1つ。番号どうしが対応している。

× NG指示

「上のほうの文字を大きくして」

→ 見出し・ロゴ・メニューのどれが変わるかわからない。

○ 伝わる指示

「ページ最上部の
見出し要素 (h1) だけ、
文字サイズを32pxにして」

AIが返すコード（例） | CSS

```
h1 {  
  font-size: 32px;  
}
```

02 タグとclass

タグは部品の種類、classはAIに場所を伝える名札。



div タグ：部品の種類（ここでは「箱」）

card class：名札。同じ名札の部品は同じ見た目になる

名札（class）で指定すれば、並び順が変わっても狙った部品だけが変わる。

× NG指示

「2つ目のカードだけ色を変えて」

→ 並び順が変わると、別のカードの色が変わってしまう。

○ 伝わる指示

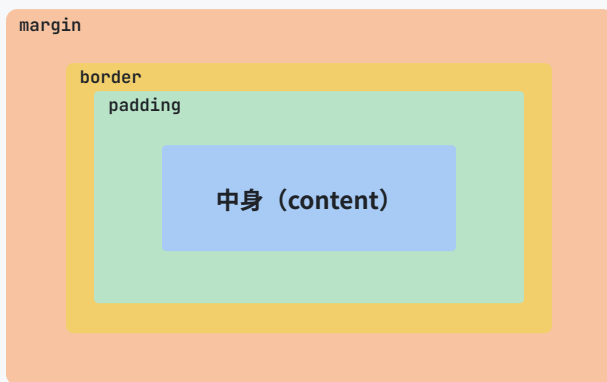
「classがcard--pickupのカードだけ、背景を薄い黄色（#fff6d6）にして」

AIが返すコード（例） | CSS

```
.card--pickup {  
  background: #fff6d6;  
}
```

03 ボックスモデル

どの要素も中身・padding・border・marginの4層の箱。



内側から 中身 → padding → border → margin の順に重なる。色は第1章で共通。

× NG指示

「もうちょっと余白を空けて」

→ 内側か外側か、どの要素の
余白かが伝わらない。

○ 伝わる指示

「カードの内側のpaddingは
20pxのまま、カード同士の
marginを24pxにして」

AIが返すコード（例） | CSS

```
.card {  
  padding: 20px; margin-bottom: 24px;  
}
```

04 padding

箱の内側、枠と中身の間のすき間。

padding: 0

padding: 16px 24px

見た目

予約する

予約する

しくみ

予約する

予約する

16px

24px

中身

padding

border

緑の部分が padding。背景色は padding の部分にも塗られる。

× NG指示

「ボタンをもう少し大きくして」

→ 文字サイズや幅まで変わり、思った形にならない。

○ 伝わる指示

「予約ボタンの文字サイズは変えずに、padding を上下 16px ・ 左右 24px に広げて」

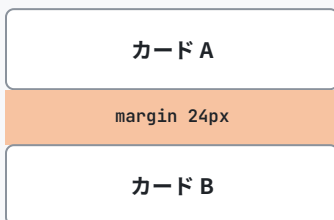
AIが返すコード（例） | CSS

```
.button {  
  padding: 16px 24px; /* 上下 左右 */  
}
```

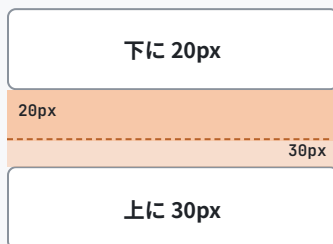
05 margin

箱の外側、となりの箱との距離。

カードの間が margin



上下の margin は重なる



間は 50px ではなく 30px

margin は透明な外側の余白。上下に並ぶと足し算されず、大きいほうだけが効く。

× NG 指示

「カードの間をもっと空けて」

→ padding が増え、カードの中がスカスカになることがある。

○ 伝わる指示

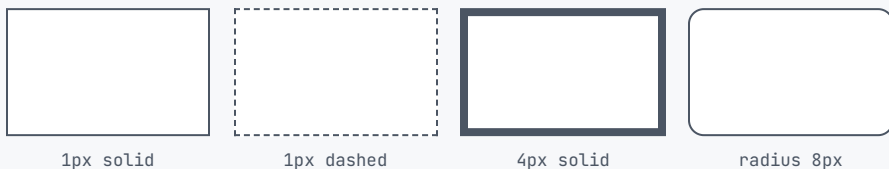
「カード同士の間隔を **margin** で 24px にして。カードの内側の余白は変えないで」

AI が返すコード（例） | CSS

```
.card + .card {  
  margin-top: 24px;  
}
```

06 border

padding と margin の境目に引く **枠線**。



border: 1px solid #d0d5dc;

太さ 線の種類 色

border は「太さ・線の種類・色」の3点セット。角の丸みは border-radius で別に指定する。

× NG指示

「カードを枠で囲って丸くして」

→ 太さ・色・丸みが
毎回ばらばらになる。

○ 伝わる指示

「カードに1pxのグレー
(#d0d5dc) のborderを
付けて、角をborder-radiusで
8px丸くして」

AIが返すコード（例） | CSS

```
.card {  
  border: 1px solid #d0d5dc; border-radius: 8px;  
}
```

07 ブロックとインライン

縦に積む箱と、文字のように横へ流れる箱。

ブロック：横幅いっぱいに広がり、縦に積まれる

季節のタルト

<h2>

旬の果物をたっぷりのせました。

<p>

カード

<div>

インライン：文字と同じように横へ流れる

営業は10時～18時です。ご予約はこれから、前日までどうぞ。

赤い点線がインライン要素（span・a・strong など）

インラインの箱には幅や高さが効かない。ボタンの形にしたいときは inline-block にする。

× NG指示

「このリンクをボタンっぽく大きくして」

→ 文字が大きくなるだけで、ボタンの形にならない。

○ 伝わる指示

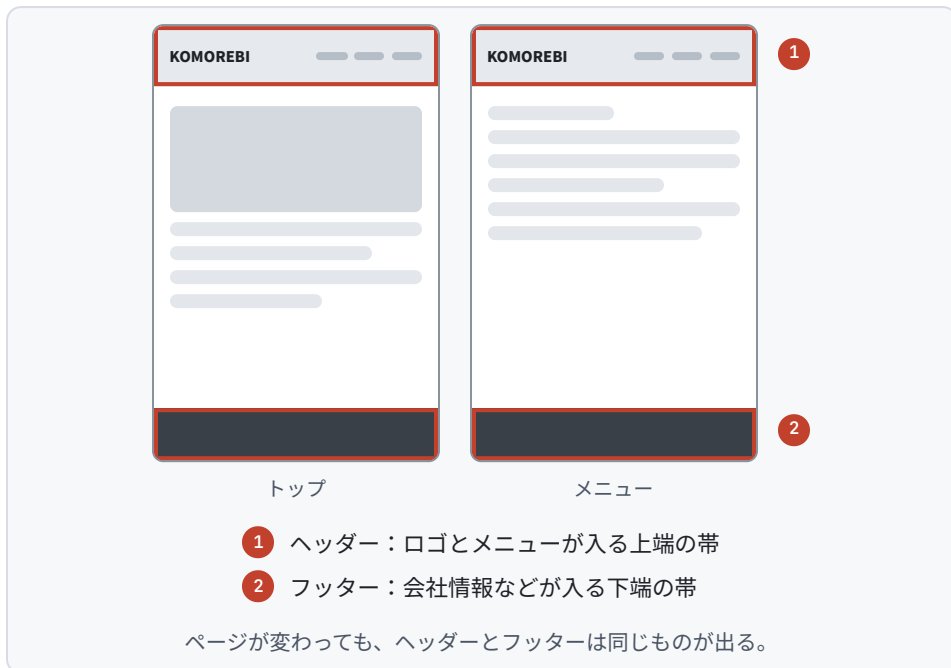
「ナビのリンク（aタグ）を inline-block にして、幅120px・高さ40pxのボタン型にして」

AIが返すコード（例） | CSS

```
nav a { display: inline-block;
width: 120px; height: 40px; }
```

08 ヘッダー・フッター

どのページにも共通で出る上端と下端の帯。



× NG指示

「一番上のところを
スクロールしても
ついてくるようにして」

→ 見出しやバナーが固定される
ことがある。

○ 伝わる指示

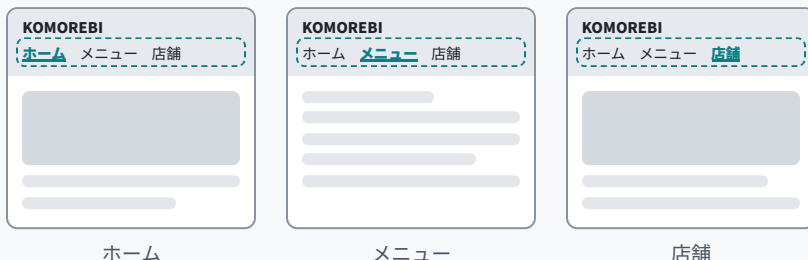
「ヘッダーをスクロールしても
画面上部に固定して。
高さは64pxで、本文が
隠れないようにして」

AIが返すコード（例） | CSS

```
header {  
  position: sticky; top: 0; height: 64px;  
}
```

09 グローバルナビ

サイトのどのページにも出る主要メニュー。



点線の中がグローバルナビ。どのページでも同じ位置にあり、今いるページの項目だけ色と下線で示す。

× NG指示

「メニューのリンクを増やして」

→ ページ内の目次やフッターのリンクが増えることがある。

○ 伝わる指示

「ヘッダーのグローバルナビの最後に「アクセス」を追加して。今いるページの項目は下線で示して」

AIが返すコード（例） | HTML

```
<nav class="global-nav">
  <a href="/menu" aria-current="page">メニュー</a> ...
  <a href="/access">アクセス</a></nav>
```

10 ハンバーガーメニュー

メニューを開くための三本線のボタン。



赤で囲んだ三本線がハンバーガー（ボタン）。開いて出てくるメニュー本体はドロワー。

× NG指示

「スマホではメニューを
三本線にして」

→ ボタンだけ作られ、押しても何も
開かないことがある。

○ 伝わる指示

「幅768px以下ではナビを
ハンバーガーメニューにま
めて。押したら右からドロワーを
開き、アイコンは×に変えて」

AIが返すコード（例） | CSS

```
@media (max-width: 768px) {  
  .global-nav { display: none; }  
  .hamburger { display: block; } }
```

11 ドロワー

画面の端から引き出しのように出てくるパネル。



右から出る（幅80%）



ドロワー：端から出る



モーダル：中央に出る

出てくる位置で呼び名が変わる。後ろを暗くする幕（オーバーレイ）はどちらにも付く。

× NG指示

「メニューを横から
シュッと出して」

→ 出てくる向き・幅・速さが
ばらばらになる。

○ 伝わる指示

「メニューを右からのドロワーに
して。幅は画面の80%、
0.3秒で開き、後ろは半透明の
黒で暗くして」

AIが返すコード（例） | CSS

```
.drawer { width: 80%; transform: translateX(100%);  
  transition: transform 0.3s; }  
.drawer.is-open { transform: translateX(0); }
```

12 パンくずリスト

今いるページがサイトのどこにあるかを示す道しるべ。

① ホーム



② メニュー

店舗



③ ケーキ（今ここ）

ドリンク

▼ 画面では1行にまとめて表示

① [ホーム](#) > ② [メニュー](#) > ③ ケーキ

見てきた順番ではなく、サイトの階層をたどる。

× NG指示

「今どこにいるかわかるやつを上に付けて」

→ 見出しや目次など、別の部品が作られる。

○ 伝わる指示

「ヘッダーの下に
パンくずリストを追加して。
ホーム>メニュー>ケーキの
順で、今いるページは
リンクにしないで」

AIが返すコード（例） | HTML

```
<nav aria-label="パンくずリスト"><ol>
  <li><a href="/">ホーム</a></li> ...
  <li aria-current="page">ケーキ</li></ol></nav>
```

13 モーダル

閉じるまで後ろの画面を操作できなくなる小窓。



- ① 小窓（モーダル本体）
- ② ×ボタン（閉じる）
- ③ オーバーレイ（後ろを暗くする幕）

× NG指示

「ボタンを押したら
ポップアップを出して」

→ 別ウィンドウや、ブラウザ標準の警告が出ることもある。

○ 伝わる指示

「予約ボタンを押したら確認の
モーダルを開いて。後ろは
半透明の黒で暗くし、×ボタンと
暗い部分のクリックで閉じて」

AIが返すコード（例） | JavaScript

```
dialog.showModal(); // モーダルとして開く
dialog.onclick = (e) => { // 暗い部分で閉じる
  if (e.target === dialog) dialog.close(); };
```

14 アコーディオン

見出しを押すと中身が開閉する折りたたみ。

① テイクアウトできますか？

+

② 予約は何日前からできますか？

—

2か月前の同じ日から、Webで予約できます。

① 駐車場はありますか？

+

① 閉じた状態：質問だけが見える（+）

② 開いた状態：答えが下に出る（—）

よくある質問（FAQ）の定番。答えは同じ場所で、質問のすぐ下に開く。

× NG指示

「よくある質問をクリックで
出るようにして」

→ 答えがモーダルや別ページで出る
ことがある。

○ 伝わる指示

「よくある質問を
アコーディオンにして。質問を
押すと答えが下に開き、最初は
全部閉じた状態にして」

AIが返すコード（例） | HTML

```
<details>
```

```
  <summary>テイクアウトできますか？</summary>
```

```
  はい、全メニューできます。</details>
```

同じ場所で中身を切り替える見出しボタン。



ページは移動せず、下の中身だけが切り替わる。ここがナビとの違い。

× NG指示

「メニューをフードとドリンクで分けて」

→ 別ページに分かれたり、見出しで縦に並んだりする。

○ 伝わる指示

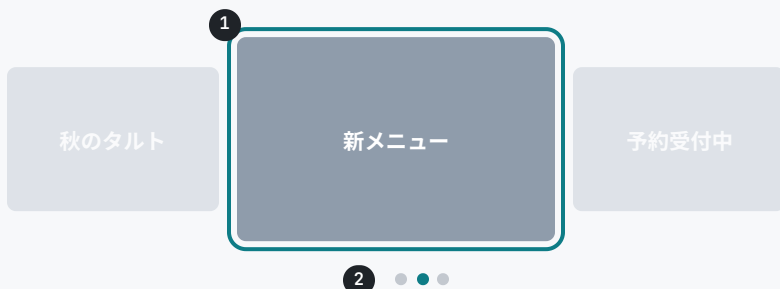
「メニュー一覧をタブで「フード／ドリンク／スイーツ」に切り替えて。ページは移動させず、最初はフードを表示して」

AIが返すコード（例） | HTML

```
<div role="tablist">
  <button role="tab" aria-selected="true">フード</button>
  <button role="tab">ドリンク</button> ...</div>
```

16 カルーセル

画像などが横に切り替わっていく表示枠。



- ① 見える枠：1枚ずつ表示し、5秒ごとに次へ
 - ② ドット：今が何枚目かを示し、押すと切り替わる
- 横に並んだ画像のうち、枠の中の1枚だけが見えている。

× NG指示

「トップの画像を
スライダーにして」

→ 音量つまみのような「スライダー」
部品が作られることがある。

○ 伝わる指示

「トップの画像3枚をカルーセルに
して。5秒ごとに自動で
次へ進め、下のドットでも
選べるようにして」

AIが返すコード（例） | JavaScript

```
new Swiper('.hero', {  
  loop: true, autoplay: { delay: 5000 },  
  pagination: { el: '.swiper-pagination' } });
```

17 トースト

操作の結果を短く知らせて自動で消える通知。



出ている間も画面は操作できる。閉じるまで操作を止めるモーダルとの違い。

× NG指示

「保存したらメッセージを
出して」

→ 閉じるまで操作できない
確認ダイアログが出ることがある。

○ 伝わる指示

「保存したら、画面の下中央に
「保存しました」のトーストを
出して。3秒で自動で消して」

AIが返すコード（例） | JavaScript

```
toast.textContent = '保存しました';  
toast.classList.add('show');  
setTimeout(() => toast.classList.remove('show'), 3000);
```

18 ファーストビュー

ページを開いてスクロールせずに見える範囲。



太枠の中がファーストビュー。画面の大きさで範囲が変わる。

× NG指示

「最初の画面をもっと
インパクトある感じに」

→ 何をどこまで変えるか伝わず、
全体が作り直される。

○ 伝わる指示

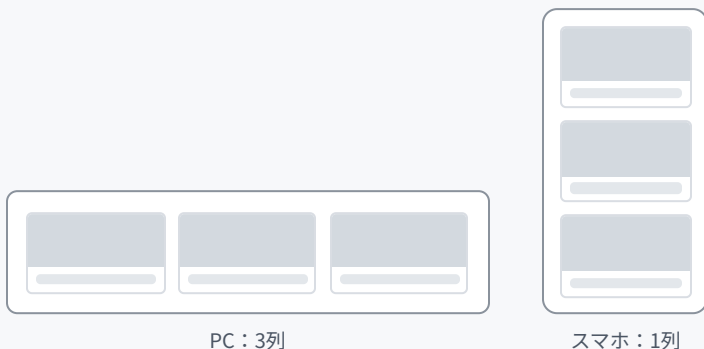
「ファーストビューを画面の
高さいっぱいにして、店名・
キャッチコピー・
予約ボタンだけを中央に置いて」

AIが返すコード（例） | CSS

```
.hero {  
  min-height: 100svh; display: grid; place-items: center;  
}
```

19 レスポンシブと ブレイクポイント

画面の幅に合わせてレイアウトを組み替えるしくみ。



画面の幅 狭い: 1列

ブレイクポイント 768px

広い: 3列

画面の幅が境目（ブレイクポイント）をまたぐと、レイアウトが切り替わる。

× NG指示

「スマホでも見やすくして」

→ 文字の縮小や一部の非表示など、
何が変わるかわからない。

○ 伝わる指示

「カードを、幅768px以上では
3列、それより狭いときは1列に
して。ブレイクポイントは
768pxの1つだけ」

AIが返すコード（例） | CSS

```
.cards { display: grid; gap: 16px; }  
@media (min-width: 768px) {  
  .cards { grid-template-columns: repeat(3, 1fr); } }
```

20 CTA

見た人にとってほしい行動へ誘うボタンやリンク。

○ 目立つのは1つ



× 全部が目立つ



CTAは1ページに1つ。ほかのボタンを控えめにする、押してほしいボタンが際立つ。

× NG指示

「ボタンを目立たせて」

→ 全部のボタンが派手になり、どれも目立たなくなる。

○ 伝わる指示

「このページのCTAは「予約する」ボタン1つだけ。メイン色で一番大きくし、ほかはグレーの枠線だけにして」

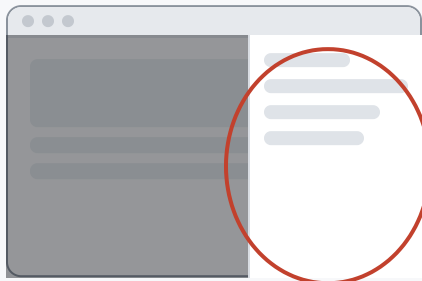
AIが返すコード (例) | CSS

```
.btn-primary { background: #0e7c86; font-size: 18px; }  
.btn { background: none; border: 1px solid #c3c8cf; }
```

知らない部品は、AIに名前を聞く

名前さえわかれば、検索も指示も一気に楽になる。

気になる部品をスクショして赤丸



AIの答え

ドロワーメニューです。

別名：オフキャンバスメニュー

英語：drawer menu

手順

- 1 気になる部品をスクショして、赤丸で囲む
- 2 AIに「名前・別名・英語名」をまとめて聞く
- 3 英語名で画像検索して、同じ部品が確かめる

○ 画像で聞く

「赤丸の部品はWebデザインで何と呼ばれますか？別名と、検索用の英語名も教えて」

○ 言葉で聞く

「画面の右端からスライドして出てくるメニュー。後ろが暗くなる。これの名前は？」

→ 場所・見た目・動きの3つを入れる

名前がわかったら、こう検索する

実物の例を見る drawer menu UI example

作り方を知る ドロワーメニュー CSS

似た部品と比べる ドロワー モーダル 違い

指示は5つの部品で組み立てる

5つがそろくと、何度頼んでも同じ答えが返ってくる。

①
どこに
場所

②
何を
部品の名前

③
どうする
見た目・動き

④
数値で
px・%・秒

⑤
変えない
守るもの

「① ヘッダーの右端に② ハンバーガーメニューを追加して。
③ タップすると右からドロワーが出るようにして、④ 幅は画面の
80%、0.3秒で開く。⑤ ロゴと文字サイズは変えないで」

違ったときの直し方

× NG指示

「なんか違う。いい
感じに直して」

→ 合っていた部分まで作り直される。

○ 伝わる指示

「ドロワーが左から
出ているので、右から出して。
ほかは変えないで」

→ 1回の指示で直すのは1か所だけ。

うまくいった指示は、ためて使い回す

- 通った指示は、部品の名前ごとにメモしておく。次のサイトでもそのまま使える
- 色・角丸・フォントなどの共通ルールは、会話の最初にまとめて伝える

メイン色 #0e7c86 / 角丸 6px / 文字 Noto Sans JP

モックで見た目を決めてから、本番に頼む

試行錯誤はモックで。本番には完成形だけを渡す。

モック用（本番から切り離した別プロジェクト）

① AIに見た目だけのモックを作ってもら



② ブラウザで見て、部品の名前で直す

▼ 見た目がOKになったら、スクショとモックのコードを持っていく

本番プロジェクト

③ スクショとコードを本番のAIに渡す



④ 今のコードのルールで実装してもらう



⑤ モックと画面を見比べて確認

○ モック用に頼む

「予約ページの見た目だけのモックを、1つのHTMLファイルで作って。本番のCSSやデータは使わず、文字は仮のものでいい」

○ 本番に頼む

「このモックのコードと画像をもとに予約ページを作って。見た目はモックどおりに、ボタンとCSSは今あるものに合わせて」

こうする理由

- 本番では、今あるCSS・ログイン・データの読み込みなどが絡み、崩れの原因が見た目なのか別の所なのかわからなくなる。モックはそれらを切り離し、**見た目だけ**を確かめられる
- 見た目の正解はモックで確定済み。本番でずれたら、原因は**本番側**にあると切り分けられる
- 本番にはスクショと一緒にモックのコードも渡す。見た目はもう実装済みなので、AIは本番に合わせて組み直すだけ。ずれたら、**組み直した所**を見ればいい